

所属类别	2026 年“华数杯”全国大学生数学建模竞赛	参赛编号
本科组		CM2602322

标题（此处换成论文的标题）

摘 要

超大规模集成电路（VLSI）中的**布图规划**是决定芯片性能、面积与良率的核心环节。本文针对 VLSI 布图规划问题，以模块位置、方向与形状为决策变量，以**模块互不重叠、不超出芯片轮廓**为约束，建立轮廓面积与总半周长线长（HPWL）等目标下的优化模型，并采用“多源初解生成—并行元启发优化—双向反馈定界校验—场景适配”四层求解框架及**模拟退火、迭代局部搜索、B*-树、约束规划（CP-SAT）**等方法求解。

针对问题一，建立**二维矩形装箱的字典序双目标优化模型**：以轮廓面积最小为一级目标、长宽比接近 1 为二级目标，并证明该问题为 NP-hard、最优解具有整数坐标性质。以 Skyline-迭代局部搜索为主算法、B*-树-模拟退火为对照求解。**结果表明**：n100、n200、n300 的轮廓面积分别为 192818、191026、291311，packing 密度达 91.97%–93.77%，长宽比均在 1.07–1.09 之间，主算法面积较 B*-树-SA 改进 0.34%–5.50%。

针对问题二，建立**固定轮廓布图规划的总半周长线长（HPWL）最小化模型**：以模块左下角坐标与 90° 旋转为决策变量，以模块互不重叠、不超出给定正方形轮廓（死区比例 $d = 0.15$ ）为约束，并利用 HPWL 目标的凸分片线性结构给出无重叠松弛的全局下界。以 Skyline-迭代局部搜索/模拟退火（ILS/SA）为主算法、B*-树-快速模拟退火与序列对-自适应遗传算法（SP-AGA）为对照求解。**结果表明**：n100、n200、n300 的总 HPWL 分别为 276865、536430、800202，均满足无重叠与轮廓可行约束，主算法较 SP-AGA 改进 22.8%–26.0%。

针对问题三，建立**最小死区比例的正方形装箱可行性判定模型**：以最小可行正方形边长 L^* 为决策量，利用可行性关于 L 的单调性在整数域上二分搜索，以启发式装箱（Skyline-ILS）给出可行证书、以 CP-SAT 给出不可行证书的双证书夹逼策略求解，并在最小轮廓下复用问题二模型更新总 HPWL。**结果表明**：n100、n200、n300 的最小死区比例分别为 11.8%、8.2%、7.1%，较问题二的 15% 显著压缩约一半；在最小轮廓下总 HPWL 分别为 282950、551698、847564，较问题二仅上升 2.2%–5.9%。

最后，我们对提出的模型进行全面的评价：本文的模型贴合实际，能合理解决提出的问题，具有实用性强，算法效率高等特点，该模型在 xx,xx,xx 方面也能使用。

关键词：关键词 1 关键词 2 关键词 3 关键词 4 关键词 5

一、问题的提出

1.1 问题的背景

布图规划 (Floorplanning) 是超大规模集成电路 (VLSI) 物理设计流程中的关键环节, 其任务是在给定芯片轮廓内确定各功能模块的位置与方向, 使模块互不重叠、不超出轮廓, 并最小化芯片面积与模块间连线长度等目标。布图质量直接影响芯片的面积、性能、良率与可靠性, 是电子设计自动化 (EDA) 领域最核心的研究问题之一。布图规划属于 NP-hard 问题, 精确算法仅适用于小规模实例, 工程上普遍采用构造式启发式与模拟退火 (SA) 等元启发式方法求解。随着大规模 ASIC/SoC 采用分层设计与固定轮廓约束, 布图规划从“最小化面积”转变为“在给定轮廓内最小化总连线长度”的约束优化问题; 半周长线长 (HPWL) 因计算简便且与真实布线长度高度正相关, 成为最常用的线长评估指标。近年来, 基于泊松方程的解析方法 (PeF) 与深度强化学习布图 (AlphaChip) 进一步拓展了大规模布图的求解范式, 为布图规划的高效求解提供了新的思路。

1.2 问题重述

(1) 问题一: 问题一要求在芯片轮廓不固定且模块之间无任何连接关系的前提下, 确定各矩形功能模块在芯片内的摆放位置与方向, 并且允许各个模块进行 90° 旋转, 使包围所有模块的**芯片轮廓面积最小**; 当轮廓面积相同时, 要求其**长宽比尽量接近 1**。需根据所建立的模型与 `.blocks` 文件信息, 对附件中的 `n100`、`n200`、`n300` 三组芯片分别独立完成模块摆放, 并给出三组芯片轮廓的面积、长宽比及模块摆放的可视化结果。

(2) 问题二: 问题二要求在**芯片轮廓固定为正方形且模块之间存在连接关系**的前提下, 确定各矩形功能模块在芯片内的摆放位置与方向, 并且允许各个模块进行 90° 旋转, 使其**模块之间总半周长线长 (HPWL) 最小**, 同时满足**模块之间互不重叠、不超出芯片轮廓**的约束条件。需根据所建立的模型与附件信息, 假设死区比例为 **0.15**, 对附件中的 `n100`、`n200`、`n300` 三组芯片分别独立完成模块摆放, 并给出三组芯片的总连线长度及模块摆放的可视化结果。

(3) 问题三: 在**问题二**的基础上, 为满足**模块互不重叠、不超出芯片轮廓**的约束条件, 求解**死区比例的最小值**, 即求解使得存在一个可行布局方案的最小死区比例 (随着死区比例减小, 芯片轮廓尺寸逐渐减小, 布图越难以满足约束条件)。需分别给出附件中 `n100`、`n200`、`n300` 三组芯片的死区比例最小值, 并基于**问题二所建立的模型**与死区比例最小值, 更新问题二的总 HPWL 及模块摆放的可视化结果。

(4) 问题四:

二、问题分析

2.1 问题一分析

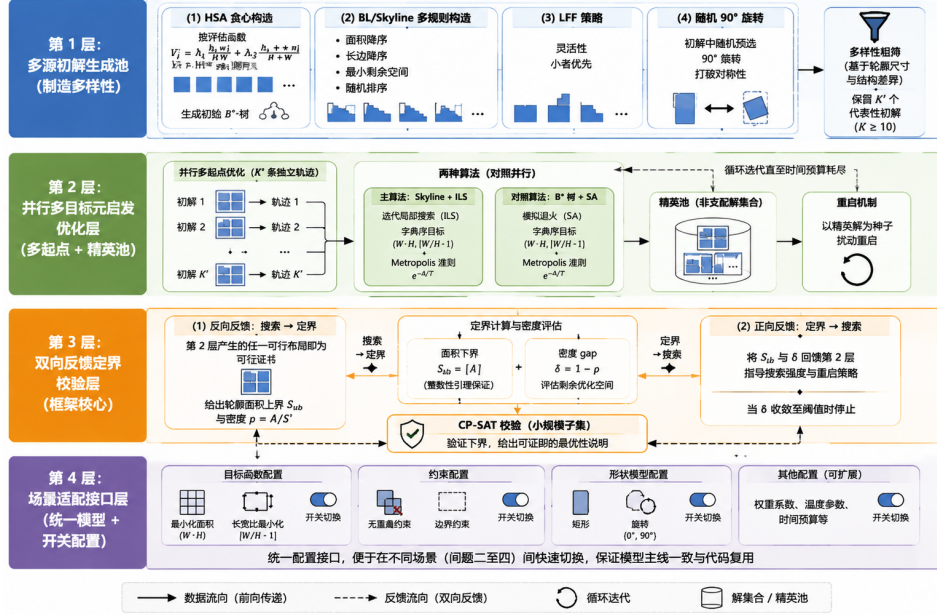


图 1 问题一四层实施框架

问题一要求在轮廓自由、模块无连接的前提下，并且允许各个矩形模块进行 90° 旋转，确定各矩形模块的位置与方向，使其芯片面积最小、面积相同时长宽比尽量接近 1。因此问题的本质在数学上属于经典的**二维矩形装箱 (Rectangle Packing)** 问题，即将给定矩形集互不重叠地摆放，并确定其最小包围盒。由于包含正方形装箱为特例，问题为 NP-hard， n 较大时无法精确求解，必须借助启发式方法。

同时，该问题存在两个优化目标：主目标为**最小化芯片轮廓面积**，次目标为**在面积相同时使长宽比尽量接近 1**。因此，采用**字典序优化策略**：先求解最优面积，再在面积最优解集中优化长宽比，严格符合题意“面积优先、长宽比次之”的要求。

基于上述分析，针对问题一制定**四层实施框架**，其结构如图 1 所示，各层环环相扣、以具体算法与数据流落地，并以实验结果闭环验证。

2.2 问题二分析

问题二要求在芯片轮廓固定为正方形、模块之间存在连接关系的前提下，确定各矩形功能模块的位置与方向，使模块之间总半周长线长 (HPWL) 最小，同时满足模块之间互不重叠、不超出芯片轮廓的约束条件。因此问题的本质在数学上属于经典的**固定轮廓布图规划 (Fixed-Outline Floorplanning)** 问题：在给定的正方形轮廓内，将矩形模块互不重叠地摆放，并最小化基于半周长线长的总连线长度。由于该问题包

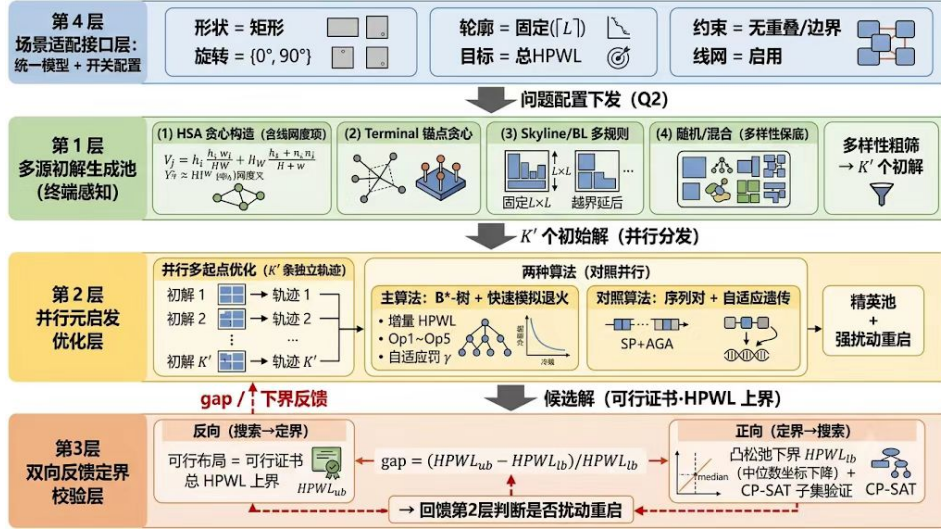


图 2 问题二四层实施框架

含二维装箱为特例，属于 NP-hard 问题， n 较大时无法精确求解，必须借助启发式方法^[5]。

同时，与问题一相比，本问的芯片轮廓不再是可以自由优化的变量，而是**给定的硬约束**：空白区从”优化目标”退化为”固定约束”，目标为单一的**最小化总 HPWL**，需与**模块互不重叠、不超出轮廓**的约束同时满足。此外，附件中 Terminal 的坐标全部位于轮廓边界，构成围绕芯片的**边界引力源**，将与之相连的模块拉向芯片四周，与不重叠约束形成对抗，这是本问优化的核心难点。

基于上述分析，针对问题二在问题一框架基础上沿用**四层实施框架**，其结构如图图 2所示：由第 4 层场景适配接口将配置切换为”轮廓固定、目标 HPWL、线网启用”，第 1-3 层分别以”终端感知多源初解、并行元启发优化（Skyline-ILS/SA 主、B*-树-快速 SA 与 SP-AGA 对照）、凸松弛下界与可行性证书的双向定界”落地，各层环环相扣、以具体算法与数据流闭环验证。

2.3 问题三分析

问题三在问题二的基础上，求解使得存在可行布局的最小死区比例 d^* ，即最小可行正方形边长 $L^* = \sqrt{A(1+d^*)}$ 。因此问题的本质在数学上属于**正方形装箱的可行性判定问题 (Square Packing Decision Problem)**：判定给定的矩形模块集能否在 $L \times L$ 的正方形轮廓内互不重叠地摆放，并在最小可行边长处寻找死区比例的临界值。该判定问题为 NP-complete， n 较大时无法精确求解，必须借助启发式方法给出可行证书、约束规划给出不可行证书。

同时，与问题一、问题二不同，本问的优化对象既不是轮廓面积也不是总连线长度，而是**可行性的临界轮廓**：随着 L 减小，空白区被压缩，模块越难以同时满足不重叠与不超出轮廓的约束。其关键性质在于可行性关于 L 的**单调性**——若 L 可行则

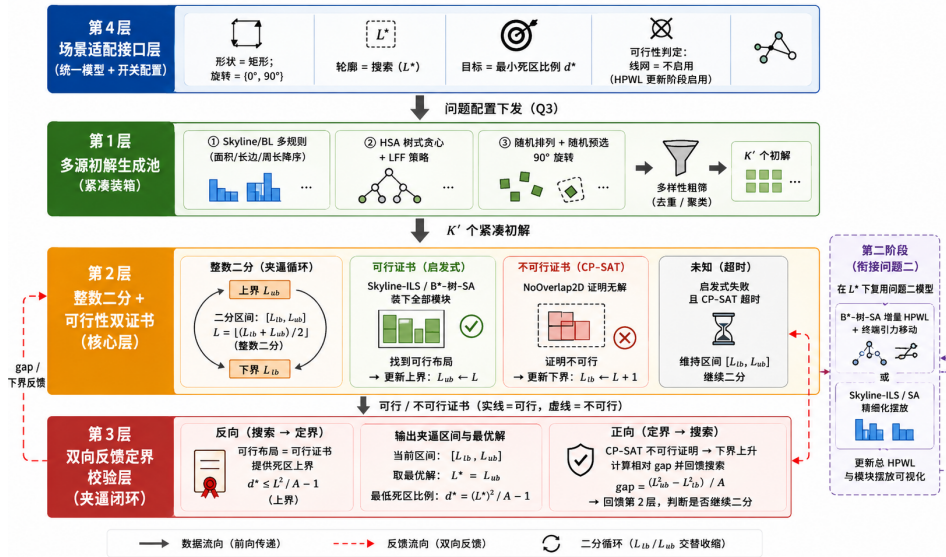


图 3 问题三四层实施框架

$L + 1$ 必然可行, 故可在整数 L 上二分搜索最小可行边长; 由于启发式只能提供可行上界、不可行下界需由 CP-SAT 的不可行证明给出, 故采用”启发式可行证书 + 约束规划不可行证书”的双证书夹逼策略, 将最小死区比例的求解转化为可证明的搜索。

基于上述分析, 针对问题三沿用四层实施框架, 其结构如图图 3所示: 由第 4 层场景适配接口将配置切换为”轮廓搜索 L^* 、目标最小死区比例、可行性判定不启用线网”, 第 1-3 层分别以”紧凑装箱多源初解、整数二分与可行性双证书、双向反馈夹逼”落地; 求得 L^* (即 d^*) 后, 再在 L^* 下复用问题二所建立的模型与算法更新总 HPWL 及模块摆放可视化, 各层环环相扣、以具体算法与数据流闭环验证。

2.4 问题四分析

....

三、模型的假设

由于……, 所以也需要遵循一定的假设

假设 1……

假设 2……

假设 3……

假设 4……

假设 5……

假设 6……

假设 7……

四、符号说明

表 1 变量符号及其解释说明

符号	解释说明
$\mathcal{B}$	硬模块 (HardBlock) 集合
$\mathcal{T}$	固定端子 (Terminal) 集合
$\mathcal{N}$	线网集合
$\mathcal{P}_k$	线网 k 的引脚集合
n	硬模块个数
w_i, h_i	模块 i 的原始宽、高
$\hat{w}_i, \hat{h}_i$	模块 i 旋转后的实际宽、高
r_i	模块 i 的旋转标志, $r_i \in \{0, 1\}$
x_i, y_i	模块 i 左下角坐标
W, H	芯片轮廓的宽、高
A	模块总面积, $A = \sum_{i \in \mathcal{B}} w_i h_i$
S	芯片轮廓面积, $S = W \cdot H$
S^*	最优轮廓面积
ρ	packing 密度, $\rho = A/S$
δ	密度 gap, $\delta = 1 - \rho$
d	死区比例
L	正方形芯片边长, $L = \sqrt{A(1+d)}$
X_t, Y_t	端子 t 的固定坐标
HPWL_k	线网 k 的半周长线长
λ_1, λ_2	HSA 评估函数权重系数
T	模拟退火温度
K, K'	初解生成数、粗筛保留数

五、模型的建立与求解

5.1 问题一

5.1.1 模型的建立

由于芯片是由各个模块拼接而来, 因此设模块集合 $\mathcal{B} = \{1, 2, \dots, n\}$, 模块 i 的原始宽高为 (w_i, h_i) ; 决策变量为模块 i 左下角坐标 (x_i, y_i) 与旋转标志 $r_i \in \{0, 1\}$, 其

中 $r_i = 1$ 表示旋转 90° ，旋转即交换宽高。旋转后模块实际宽高为

$$\hat{w}_i = (1 - r_i)w_i + r_i h_i, \quad \hat{h}_i = r_i w_i + (1 - r_i)h_i \quad (1)$$

设芯片轮廓宽、高分别为 W, H ，模块总面积为 $A = \sum_{i \in \mathcal{B}} w_i h_i$ ，芯片轮廓总面积为 $S = W \cdot H$ 。

又因为在芯片制作是要求模块互不重叠，即 $\forall i, j \in \mathcal{B}, i < j$ ，则下述四个条件至少满足其一：

$$(x_i + \hat{w}_i \leq x_j) \vee (x_j + \hat{w}_j \leq x_i) \vee (y_i + \hat{h}_i \leq y_j) \vee (y_j + \hat{h}_j \leq y_i) \quad (2)$$

轮廓边界约束：

$$0 \leq x_i, \quad x_i + \hat{w}_i \leq W, \quad 0 \leq y_i, \quad y_i + \hat{h}_i \leq H, \quad \forall i \in \mathcal{B} \quad (3)$$

上述非重叠约束与轮廓边界约束的几何含义如图图 4所示：相邻模块在水平或竖直方向上必须保持分离以保证互不重叠，同时所有模块整体必须位于宽为 W 、高为 H 的芯片轮廓之内。

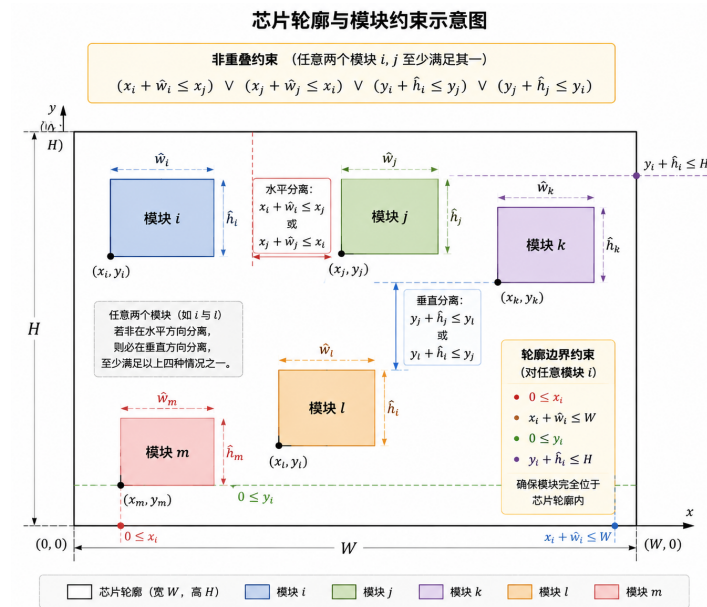


图 4 问题一约束条件示意图

在布图规划中，评价布局优劣的首要指标是芯片轮廓面积。由于题目要求在面积相同的情况下长宽比越接近 1 越好，两个目标之间存在明确的主次关系，不宜直接加权求和，故本文采用**字典序优化策略**：先最小化轮廓面积 $S = W \cdot H$ ，即

$$\min f_1 = W \cdot H \quad (4)$$

在得到最优面积 S^* 后，再在面积最优解集中进一步优化长宽比，即

$$\min f_2 = \left| \frac{W}{H} - 1 \right| \quad \text{s.t. } WH = S^* \quad (5)$$

其中 S^* 为式((4) 式)的最优值, 上述两式构成字典序双目标, 严格符合题意” 面积优先、长宽比次之” 的要求。

5.1.2 理论分析

为给求解算法的设计提供严谨的理论依据, 本小节从计算复杂性、解空间结构与性能界三个层面, 对问题一的模型性质进行分析, 依次给出 NP-hard 性质、整数性引理与面积下界三个结论, 并据此确定” 启发式为主、精确法为辅” 的求解策略。

(1) **NP-hard 性质**。该问题包含**经典二维矩形装箱 (Rectangle Packing)** 问题为特例。由于将正方形装箱判定问题可多项式归约到本问题, 故本问题属于 NP-hard 问题。这意味着当模块规模 n 较大时, 问题的解空间随 n **呈阶乘级爆炸**, 精确算法 (如 MILP、约束规划) 在最坏情况下需要指数级时间, 仅适用于小规模实例。因此在 $n = 100 \sim 300$ 的赛题规模下, 无法依赖精确算法在合理时间内得到最优解, 必须采用启发式与元启发式方法进行求解^[1]。这一结论直接决定了后续求解框架的总体方向, 即第 1 层的多源初解构造与第 2 层的元启发式改进。

(2) **整数性引理**。由**压缩 (compaction) 性质**可知, 对任意可行布局, 总可以依次将所有模块向左、向下压缩至无法再移动, 而这一压缩过程不会破坏可行性。压缩后, 每个模块的坐标必等于其左侧 (或下方) 若干模块的宽度 (高度) 之和, 由于所有模块尺寸均为整数, 故压缩后各模块坐标必为整数。由此可得: 必然存在一个整数坐标的最优解, 即最优轮廓尺寸 W^*, H^* 均为整数^[2]。这一性质将连续坐标的搜索空间离散为整数格点, 使轮廓尺寸的搜索可限制在整数域上, 从而为第 3 层中” 在整数 S 的分解上枚举并检验可 packing 性” 的定界校验提供了理论支撑。

(3) **下界与密度**。由于各模块互不重叠且全部位于轮廓内部, 模块总面积 $A = \sum_{i \in \mathcal{B}} w_i h_i$ 必然不超过轮廓面积, 故面积最优值满足下界 $S^* \geq \lceil A \rceil$ 。据此定义 packing 密度 $\rho = A/S$ ($0 < \rho \leq 1$), 并以 $\text{gap} = 1 - \rho$ 衡量当前解与面积下界的差距: 密度越接近 1, 说明空白面积越少, 布局质量越高。该指标既可用于求解过程中实时判断剩余优化空间, 也可用于最终结果的量化评价, 使启发式求解结果的优劣具有可度量的客观标准。

综上, NP-hard 性质决定了求解必须采用启发式方法, 整数性引理将搜索空间离散化并提供了定界枚举的依据, 面积下界与密度指标则为求解过程的迭代控制与最终结果的质量评价提供了量化准则。三者共同构成问题一求解框架的” 搜索方向—搜索空间—搜索质量” 理论闭环, 为后续四层求解框架的设计奠定了完整的基础。

5.1.3 模型的求解

针对问题 NP-hard 与 $n = 100 \sim 300$ 的大规模特点, 本文采用图 1 所示的四层求解框架, 各层以具体算法与数据流落地。

第 1 层：多源初解生成池，制造多样性。

为了丰富初始解的多样性，该论文通过综合四种机制生成 $K \geq 10$ 个初始布局：

- (1) **HSA 贪心构造**：按评估函数 $V_i = \lambda_1 h_i w_i / (HW) + \lambda_2 (h_i + w_i) / (H + W)$ 降序装箱生成紧凑的初始 B*-树^[1]；
- (2) **BL/Skyline 多规则构造**：分别按面积降序、长边降序、最小剩余空间与随机排序四种优先级各生成布局；
- (3) **LFF 策略**；
- (4) **初解中随机预选 90° 旋转以打破对称性**。

随后以轮廓尺寸与结构差异构造多样性指标，粗筛保留 K' 个代表性初解进入下一层。

第 2 层：并行多目标元启发优化层，多起点结合精英池。

在第一层中的 K' 个初始解传入第二层之后，对 K' 个初解各启动一条独立的元启发轨迹，并多核并行，以 **Skyline 结合迭代局部搜索 (ILS) 为主算法**、B*-树-模拟退火 (SA) 为对照算法^[3]；两者均采用字典序目标 $(W \cdot H, |W/H - 1|)$ ，并按 **Metropolis 准则** $e^{-\Delta/T}$ 接受暂时劣解以跳出局部最优。各轨迹收敛后将解放入精英池，再以精英解为种子扰动重启，形成“多起点—精英池—重启”循环直至时间预算耗尽。

第 3 层：双向反馈定界校验层。由于只使用启发式或精确算法都存在缺点：

- 启发式算法只能给出一个可行解，但是无法判断是否为最优解；
- 精确算法可以给出最优解，但是在大规模问题上求解时间过长，无法在合理时间内得到结果。

因此该论文通过**双反馈的方式**实现定界与搜索的双向闭环：

- (1) **反向反馈 (搜索 $\rightarrow$ 定界)**：第 2 层产生的任一可行布局即为**可行证书**，给出轮廓面积上界 S_{ub} 与密度 $\rho = A/S$ ；
- (2) **正向反馈 (定界 $\rightarrow$ 搜索)**：由整数性引理与面积下界 $S_{lb} = \lceil A \rceil$ ，实时计算当前解与下界的密度 gap $\delta = 1 - \rho$ 并回馈第 2 层，判断剩余优化空间；当 δ 收敛至阈值时停止，并以 CP-SAT 在小规模子集上验证下界、给出可证明的最优性说明。

第 4 层：场景适配接口层。

因为接下来的问题都需要在该框架上进行，所以该论文将问题一抽象为“目标函数（面积/长宽比）、约束（无重叠/边界）与形状模型（矩形 + 旋转 $\{0^\circ, 90^\circ\}$ ）”的统一配置，第 1-3 层代码仅通过开关切换适配，保证问题二至四的模型主线一致与代码复用。

算法参数设置。本文各算法关键参数如表表 2 所示，其中模拟退火的初温与降温速率按目标尺度自适应整定，HSA 权重与初解池规模经小规模预实验确定。

5.1.4 求解结果与分析

基于上述四层框架，对附件中 n100、n200、n300 三组芯片独立求解，得到各芯片轮廓的最优摆放结果如表表 3 所示。其中 A 为模块总面积， $\rho = A/S$ 为 packing 密

表 2 问题一求解算法关键参数

层/算法	参数	取值
第 1 层初解生成	初解数 K / 保留数 K'	10 / 5
	HSA 评估权重 (λ_1, λ_2)	$(0.6, 0.4)$ 、 $(0.35, 0.65)$ 、 $(0.8, 0.2)$
	随机预旋转概率 p_{rot}	0.3
	初解条带宽	$\lceil \sqrt{A} \rceil$
第 2 层 Skyline-ILS	面积阶段初温 T_0	400
	长宽比精调阶段初温 T_0	5×10^{-3}
	降温速率 α	0.9 (每 300 次迭代)
	温度下限 / 重启周期	$0.02T_0$ / 900 次迭代
	扰动规模 k	$1 \sim \min(6, \lfloor n/12 \rfloor + 2)$
	强扰动 (精英池重启)	重排 $\lfloor n/5 \rfloor$ 个、翻转 30% 旋转、30% 逆序
	条带宽网格	$0.92\sqrt{A} \sim 1.35\sqrt{A}$, 6 档
	时间预算 (ILS/长宽比/重启)	30/ 12/ 15 s
第 2 层 B*-树-SA	初温 t_0	500
	降温速率 α (每 100 次迭代)	0.995
	最低温 $t_{\min}$	10^{-4}
	时间预算	20 s
第 3 层 CP-SAT	子集规模 / 时间上限	15 / 60 s
	字典序目标	面积放大系数 10^7

度, $\text{gap} = 1 - \rho$ 为与面积下界的相对差距, 长宽比取 $\max(W, H) / \min(W, H)$ 。

表 3 问题一三组芯片的求解结果

数据集	n	轮廓尺寸 $W \times H$	轮廓面积 S	密度 $\rho(\text{gap})$
n100	100	421×458	192818	93.09%(6.91%)
n200	200	418×457	191026	91.97%(8.03%)
n300	300	523×557	291311	93.77%(6.23%)

(1) **求解质量分析**。三组芯片的求解结果均为可行布局, packing 密度分别达到 **93.09%、91.97% 和 93.77%**, 即各轮廓内约 92% 以上的面积被模块有效利用, 仅约 6%–8% 的空白面积作为间隙散布于模块之间。对比面积下界 $S_{lb} = \lceil A \rceil$, 三组结果的 gap 均在 6%–8% 之间, 说明该四层框架已逼近矩形装箱问题的理论极限; 由于任意两个整数尺寸矩形在一般情形下无法实现面积利用率 100% 的完美铺砌, 这一 gap 水平符合矩形 packing 的理论预期。同时, 三组结果的长宽比分别为 1.088、1.093 和 1.065, 均接近 1, 满足题目“面积相同时长宽比尽量接近 1”的次目标要求。

(2) **算法对比分析**。为进一步验证四层框架 (主算法 Skyline-ILS) 的求解优势, 将

其与对照算法 B*-树-SA 的求解结果进行对比，如表表 4所示。

表 4 主算法与 B*-树-SA 对照算法的轮廓面积对比

数据集	Skyline-ILS 面积	B*-树-SA 面积	面积差	改进比例
n100	192818	193479	661	0.34%
n200	191026	195390	4364	2.23%
n300	291311	308313	17002	5.50%

由表表 4可见，主算法在三组数据上的轮廓面积均小于 B*-树-SA，且改进比例随模块规模增大而明显上升（0.34%→2.23%→5.50%）。这得益于四层框架的”多源初解-多起点精英池”策略：规模越大，解空间越复杂，单一退火轨迹越容易陷入局部最优，而多起点并行与扰动重启机制能更充分地探索解空间，从而获得显著更优的布局。

(3) 收敛性分析。三组芯片求解过程的收敛曲线如图图 5所示。可以看出，三组曲线均呈”快速下降—缓慢逼近”的典型收敛形态：退火初期轮廓面积快速下降，搜索效率高；随着温度降低，改进幅度逐渐放缓并趋于平稳，最终收敛到稳定的最优值附近，说明四层框架的降温策略与精英池重启机制设计合理，能够在有限时间内收敛到高质量解。

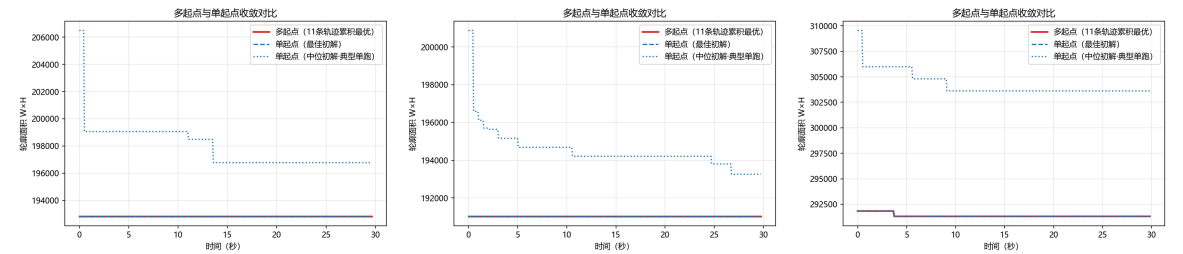


图 5 问题一三组芯片的求解收敛曲线

三组芯片的模块摆放可视化结果如图图 6所示，图中矩形代表各功能模块，其左下角坐标即模块摆放位置，红色虚线框为芯片轮廓。由图可见，各模块均被紧密、无重叠地排布于轮廓之内，大模块与中模块相互嵌合，空白间隙分散且细小，直观验证了求解结果的可行性与紧凑性。

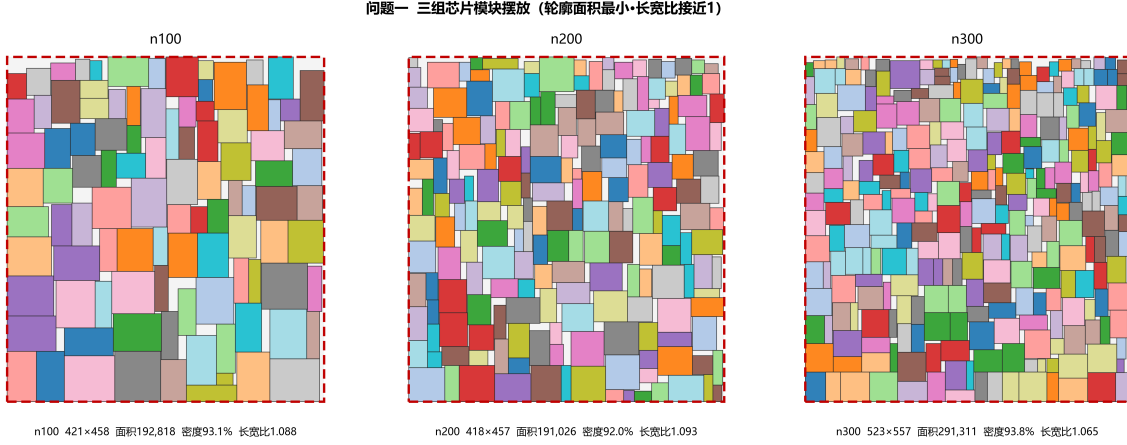


图 6 问题一三组芯片的模块摆放可视化

5.2 问题二

5.2.1 模型的建立

由于模块之间的连接关系与 Terminal 固定引脚会共同影响模块的摆放，因此设模块集合 $\mathcal{B} = \{1, 2, \dots, n\}$ ，模块 i 的原始宽高为 (w_i, h_i) ；决策变量为模块 i 左下角坐标 (x_i, y_i) 与旋转标志 $r_i \in \{0, 1\}$ ，其中 $r_i = 1$ 表示旋转 90° ，旋转即交换宽高。旋转后模块实际宽高为

$$\hat{w}_i = (1 - r_i)w_i + r_i h_i, \quad \hat{h}_i = r_i w_i + (1 - r_i)h_i \quad (6)$$

模块引脚位于几何中心，故模块 i 的引脚坐标为

$$c_i = \left(x_i + \frac{\hat{w}_i}{2}, y_i + \frac{\hat{h}_i}{2} \right) \quad (7)$$

设 Terminal 集合为 $\mathcal{T}$ ，端子 t 的固定坐标为 (X_t, Y_t) ；线网集合为 $\mathcal{N}$ ，线网 k 的引脚集合为 $\mathcal{P}_k$ （由模块中心与 Terminal 固定坐标构成）。芯片轮廓为固定正方形，边长由式 (1) 确定，即

$$L = w_f = h_f = \sqrt{A(1 + d)}, \quad A = \sum_{i \in \mathcal{B}} w_i h_i, \quad d = 0.15 \quad (8)$$

模块互不重叠约束与问题一相同，即 $\forall i, j \in \mathcal{B}, i < j$ ，下述四个条件至少满足其一：

$$(x_i + \hat{w}_i \leq x_j) \vee (x_j + \hat{w}_j \leq x_i) \vee (y_i + \hat{h}_i \leq y_j) \vee (y_j + \hat{h}_j \leq y_i) \quad (9)$$

与问题一不同的是，芯片轮廓宽度与高度不再是决策量，而是固定的正方形边长 L ，故轮廓边界约束为：

$$0 \leq x_i, \quad x_i + \hat{w}_i \leq L, \quad 0 \leq y_i, \quad y_i + \hat{h}_i \leq L, \quad \forall i \in \mathcal{B} \quad (10)$$

Terminal 不占用芯片面积, 不参与面积、重叠与边界约束, 允许与模块坐标点重叠, 仅以固定坐标作为引脚参与线长计算。

对线网 $k \in \mathcal{N}$, 记其引脚坐标的横纵最大、最小值为 $X_k^{\max} = \max_{p \in \mathcal{P}_k} X_p$ 、 $X_k^{\min} = \min_{p \in \mathcal{P}_k} X_p$ (Y 轴同理), 则其半周长线长为

$$\text{HPWL}_k = (X_k^{\max} - X_k^{\min}) + (Y_k^{\max} - Y_k^{\min}) \quad (11)$$

本文的目标为在约束((9) 式)与((10) 式)下, 最小化所有线网的 HPWL 之和, 即

$$\min_{x,y,r} \sum_{k \in \mathcal{N}} \text{HPWL}_k \quad (12)$$

目标函数可线性化: 对每条线网引入辅助变量 $X_k^{\max} \geq X_p$ 、 $X_k^{\min} \leq X_p$ (Y 轴同理), 在最小化目标下辅助变量自动取紧; 式((9) 式)采用大 M 法线性化后, 式((12) 式)与约束构成标准 MILP 形式, 可供精确求解器使用。

5.2.2 理论分析

为给求解算法的设计提供理论依据, 本小节从目标函数结构与下界两个层面对问题二的模型性质进行分析 (计算复杂性已在问题一中讨论, 固定轮廓布图规划同样为 NP-hard, 此处不再赘述)。

(1) 目标函数的凸分片线性结构。 HPWL_k 关于模块中心坐标是凸的分片线性函数 ($\max$ 与 $-\min$ 均为凸函数), 故总目标 $\sum_k \text{HPWL}_k$ 为凸函数。该性质带来两点直接推论: 其一, 去掉非重叠约束((9) 式)后的”解析松弛”问题为凸优化问题, 可高效求解并给出全局下界; 其二, 在固定模块间相对几何关系 (packing 模式) 下, 模块最优位置可由坐标下降/中位数更新求得, 为启发式搜索中”终端引力移动”等邻域操作提供理论依据。

(2) 松弛下界。 去掉非重叠约束后约束集合放宽, 得到凸松弛问题

$$\text{HPWL}_{lb} = \min_{x,y,r} \sum_{k \in \mathcal{N}} \text{HPWL}_k \quad \text{s.t. ((10) 式)} \quad (13)$$

其最优值满足 $\text{HPWL}_{lb} \leq$ 原问题最优值, 构成原问题的全局下界; 据此可计算启发式解与下界的相对 gap, 定量刻画求解质量。三组数据的松弛下界实测值在求解结果一节给出。

5.2.3 模型的求解

针对问题二的固定轮廓与 HPWL 目标, 第 1 层生成终端感知的多源初解池 (网络度加权树式贪心、Terminal 锚点、松弛中心排序、随机混合); 第 2 层以 **Skyline-ILS/SA** 为主算法, 将固定轮廓约束内建于装箱、在可行域内直接优化 HPWL, 并以 **B*-树-快速模拟退火** (增量 HPWL+ 自适应罚权重 γ + 终端引力移动) 与序列对-自适

应遗传算法 (SP-AGA) 为对照算法, 最后以终端引力合法化 (PeF 范式) 对主解做后处理精调; 第 3 层以凸松弛下界给出全局下界并回馈 gap。各算法关键参数如表表 5 所示。

表 5 问题二求解算法关键参数

层/算法	参数	取值
第 1 层初解生成	保留初解数 K'	5
	HSA 权重 $(\lambda_1, \lambda_2, \lambda_3)$	(0.5, 0.2, 0.3)、(0.4, 0.3, 0.3)
	随机预旋转概率 p_{rot}	0.3
	分簇网格 g	Terminal 锚点 $\lceil L/4 \rceil$ 、松弛中心 $\lceil L/5 \rceil$
第 2 层 Skyline-ILS/SA	初温 T_0 (自适应)	$2\lceil \Delta \rceil + 1$ (30 次探测)
	降温速率 α	0.96 (每 $\max(50, 3n)$ 次迭代)
	最低温 $T_{\min}$	10^{-2}
	扰动三档 (轻/中/重)	$k=n/20$ 、 $k=n/10$ 、 $k=n/6$
	扰动选择概率	重 8%、轻 30%、其余为中
	停滞重启阈值	400 次无改进 $\rightarrow T=0.4T_0$
	时间预算 (ILS/重启)	30/ 15 s
	时间预算 (ILS/重启)	30/ 15 s
第 2 层 B*-树-SA	初温 t_0	$\max(500, (\text{HPWL} + \gamma \cdot \text{ovf})/n)$
	降温速率 α / 最低温	0.995 (每 400 次) / 10^{-2}
	罚权重 γ	$\gamma_0=5 \times 10^5$, 范围 $[10^3, 10^8]$
	γ 自适应	每 300 次接受: 可行占比 $> 85\% \rightarrow \times 0.7$ 、 $< 45\% \rightarrow \times 2.0$
	终端引力概率 p_{grav}	0.25
	时间预算	20 s
第 2 层 SP-AGA	种群 / 精英保留	24/ 4
	罚权重 γ	5×10^5
	交叉/变异率 (自适应)	$p_c=0.9 - 0.3s$ 、 $p_m=0.1 + 0.2s$
	时间预算	30 s
PeF 合法化精调	轮数 / 每轴候选步数	10/ 10
	温度 (每轮 $\times 0.85$)	$\max(10, 0.02\text{HPWL}/n)$
	时间预算	20 s
第 3 层凸松弛下界	坐标下降迭代上限 / 容差	300/ 10^{-6}

5.2.4 求解结果与分析

基于上述模型与四层框架, 对附件中 n100、n200、n300 三组芯片独立求解, 得到各芯片轮廓内总 HPWL 最小的模块摆放结果如表表 6 所示。其中 L 为公式 (1) 向上取整后的轮廓边长 ($d = 0.15$), HPWL_{lb} 为凸松弛下界, gap 定义为 $\text{HPWL}/\text{HPWL}_{lb} - 1$ 。

(1) 求解质量分析。三组芯片的求解结果均为可行布局, 即所有模块互不重叠且完全位于 $L \times L$ 轮廓之内。由表表 6 可见, 总 HPWL 随模块规模增大而显著上升

表 6 问题二三组芯片的求解结果

数据集	n	轮廓 $L \times L$	总 HPWL	松弛下界 HPWL _{lb}	gap	是否可行
n100	100	455×455	276865	111038	149.3%	是
n200	200	450×450	536430	185214	189.6%	是
n300	300	561×561	800202	233375	242.9%	是

(276865 $\rightarrow$ 536430 $\rightarrow$ 800202)，与模块数及线网规模的增长趋势一致。与凸松弛下界相比，三组结果的总 HPWL 约为下界的 2.5–3.4 倍，其原因在于：松弛下界去除了非重叠约束，允许各模块在轮廓内自由移动至其相连引脚的中位数位置；而真实布局中模块必须互不重叠，并被边界 Terminal 的引力牵制，难以同时逼近各自的中位数理想位置，故 HPWL 必然显著高于下界。同时，gap 随规模增大而上升 (149.3% $\rightarrow$ 189.6% $\rightarrow$ 242.9%)，说明模块越多、几何约束对线长的制约越强，也验证了以凸松弛下界刻画“几何约束代价”的合理性。

(2) 算法对比分析。为进一步验证主算法 Skyline-ILS/SA（将固定轮廓约束内建于装箱、在可行域内直接优化 HPWL）的求解优势，将其与对照算法 B*-树-快速模拟退火及序列对-自适应遗传算法 (SP-AGA) 的求解结果进行对比，如表表 7 所示。

表 7 主算法与对照算法的总 HPWL 对比

数据集	主算法 HPWL	B*-树-SA	SP-AGA	较 SP-AGA 改进
n100	276865	294720	365988	24.4%
n200	536430	540688	695055	22.8%
n300	800202	812897	1081840	26.0%

由表表 7 可见，主算法在三组数据上的总 HPWL 均低于两种对照算法，较 SP-AGA 改进 22.8%–26.0%，较 B*-树-SA 亦改进 0.8%–6.1%。这得益于 Skyline 将固定轮廓约束内建于装箱，每次评估均为可行解、不浪费预算修复越界，配合增量 HPWL 评估与终端引力移动在可行域内高效压低线长；B*-树只能表示“左紧致”类 packing，且从不可行货架种子出发需花费迭代修复越界，解质量次之；SP-AGA 因表示解码与适应度评估开销大、收敛较慢，在相同时间预算下得到的解质量最差。

(3) 收敛性分析。三组芯片求解过程的收敛曲线如图图 7 所示。可以看出，三组曲线均呈“快速下降—缓慢逼近”的典型收敛形态：退火初期总 HPWL 快速下降，搜索效率高；随着温度降低，改进幅度逐渐放缓并趋于平稳，最终收敛到稳定值附近，说明四层框架的降温策略与精英池重启机制设计合理，能够在有限时间内收敛到高质量解。

三组芯片的模块摆放可视化结果如图图 8 所示，图中矩形代表各功能模块，其左

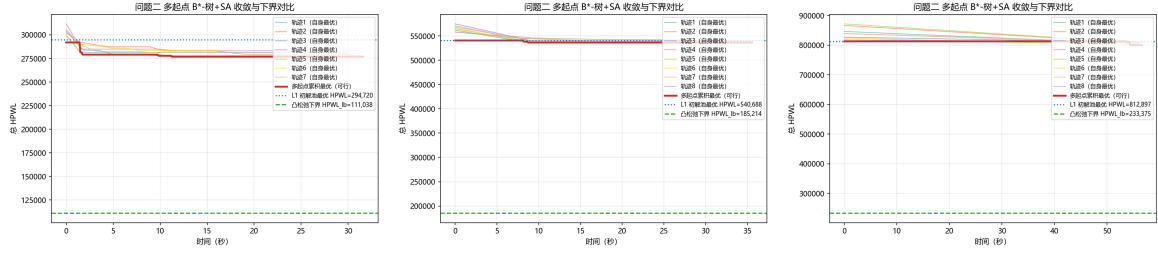


图 7 问题二三组芯片的求解收敛曲线

下角坐标即模块摆放位置，红色虚线框为 $L \times L$ 的芯片轮廓。由图可见，各模块均被紧密、无重叠地排布于轮廓之内，且模块的分布明显受到边界 Terminal（图中红点）的影响，直观反映了”边界引力”作用下对总 HPWL 的优化结果。

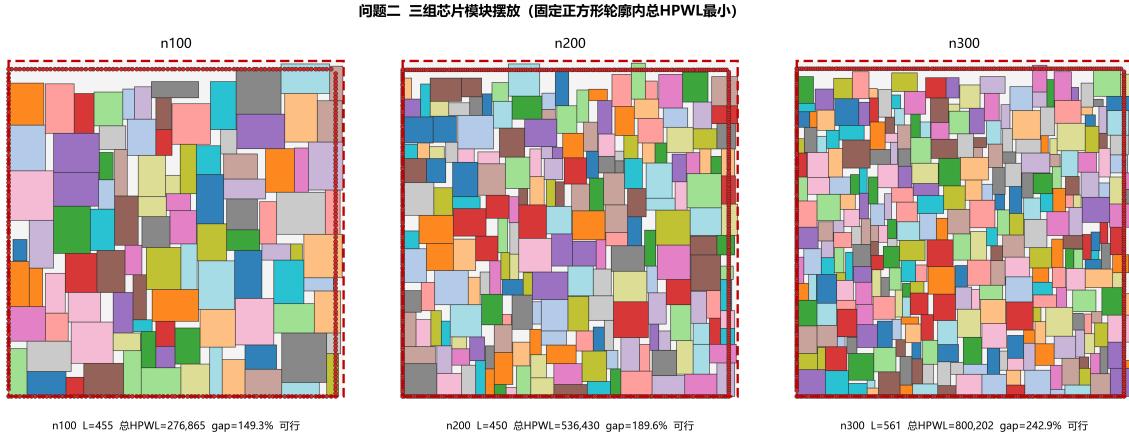


图 8 问题二三组芯片的模块摆放可视化

5.3 问题三

5.3.1 模型的建立

由于问题三要求在问题二的基础上求解满足约束的最小死区比例，因此设模块集合 $\mathcal{B} = \{1, 2, \dots, n\}$ ，模块 i 的原始宽高为 (w_i, h_i) ；决策变量为模块 i 左下角坐标 (x_i, y_i) 与旋转标志 $r_i \in \{0, 1\}$ ，其中 $r_i = 1$ 表示旋转 90° ，旋转即交换宽高。旋转后模块实际宽高为

$$\hat{w}_i = (1 - r_i)w_i + r_i h_i, \quad \hat{h}_i = r_i w_i + (1 - r_i)h_i \quad (14)$$

与问题二不同，芯片轮廓边长 L 不再是给定常数，而是待求解的决策量，与死区比例 d 通过公式 (1) 一一对应，即 $L = \sqrt{A(1 + d)}$ 。模块互不重叠约束与前述相同，即 $\forall i, j \in \mathcal{B}, i < j$ ，下述四个条件至少满足其一：

$$(x_i + \hat{w}_i \leq x_j) \vee (x_j + \hat{w}_j \leq x_i) \vee (y_i + \hat{h}_i \leq y_j) \vee (y_j + \hat{h}_j \leq y_i) \quad (15)$$

轮廓边界约束为：

$$0 \leq x_i, \quad x_i + \hat{w}_i \leq L, \quad 0 \leq y_i, \quad y_i + \hat{h}_i \leq L, \quad \forall i \in \mathcal{B} \quad (16)$$

称 L 可行，当且仅当存在 (x, y, r) 同时满足式((15) 式)与式((16) 式)。Terminal 不占用面积、不参与重叠与边界约束，故在可行性判定中无需考虑。

本文的目标为求解最小可行正方形边长，进而得到最小死区比例，即

$$L^* = \min \left\{ L \in \mathbb{Z}_{>0} : \exists \text{ 满足式((15) 式)与式((16) 式)的摆放} \right\}, \quad d^* = \frac{(L^*)^2}{A} - 1 \quad (17)$$

求得 L^* 后，还需在 $L = L^*$ 下复用问题二的模型最小化总 HPWL，即

$$\min_{x,y,r} \sum_{k \in \mathcal{N}} \text{HPWL}_k \quad \text{s.t. ((15) 式), ((16) 式), } L = L^* \quad (18)$$

式((17) 式)与式((18) 式)构成字典序双目标：先求最小死区比例，再在最小轮廓内优化线长，与题目”基于问题二模型和死区比例最小值更新总 HPWL”的要求一致。

5.3.2 理论分析

为给求解算法的设计提供理论依据，本小节从计算复杂性、单调性与下界三个层面对问题三模型性质进行分析。

(1) NP-complete (决策版本)。判定”给定的矩形模块集能否在 $L \times L$ 正方形内无重叠摆放”是经典正方形装箱决策问题，为 NP-complete；在 $n = 100 \sim 300$ 的规模下，精确判定无法在时限内闭合，必须借助启发式给出可行证书、约束规划给出不可行证书，这决定了本问”双证书夹逼”的求解方向。

(2) 可行性关于 L 的单调性。若 L 可行，则 $L + 1$ 必然可行——因为 L 下的任一可行摆放整体不变地落在 $(L + 1) \times (L + 1)$ 内仍满足式((15) 式)与式((16) 式)。因此可行性在整数 L 上单调，可在整数域上二分搜索最小可行边长，这是本问高效求解的关键性质。

(3) 整数性引理与下界。由问题一的压缩论证，存在整数坐标的最优可行解，故 L^* 为整数，二分域可限制在整数上。同时，模块总面积 A 给出边长下界

$$L^* \geq L_{lb} = \max \left(\lceil \sqrt{A} \rceil, \max_i \max(w_i, h_i) \right), \quad d^* \geq d_{lb} = \frac{L_{lb}^2}{A} - 1 \quad (19)$$

n100、n200、n300 的下界实测值分别为 $L_{lb} = 424, 420, 523$ ，对应 $d_{lb} \approx 0.153\%, 0.401\%, 0.131\%$ ；该下界假定面积利用率 100%，一般不可达，仅作夹逼下界。结合启发式给出的可行上界，可构成 $[L_{lb}, L_{ub}]$ 的夹逼区间并据此二分。

5.3.3 模型的求解

针对问题三，在问题一/二框架基础上采用**整数二分 + 可行性双证书**策略求解最小可行边长 L^* 。利用可行性关于 L 的单调性，在整数区间 $[L_{lb}, L_{ub}]$ 上二分；每层调用**可行性 oracle**：以 Skyline 装箱多源初解与 Skyline-ILS（将越界量压至 0）给出**可行证书**（ L 可行则收缩上界），以 CP-SAT ‘NoOverlap2D’对面积最大的前 15 个模块给出**不可行证书**（子集不可行则整体不可行、抬升下界），启发式失败且 CP-SAT 超时则维持区间继续二分；并采用**增量收缩 warm start**，将上一可行 L 的解（放置顺序 + 旋转）原样作为下一更小 L 的初始解，避免逐层重复搜索。区间收敛后取 $L^* = L_{ub}$ 并附可行布局证书， $d^* = (L^*)^2/A - 1$ 。求得 L^* 后，在 $L = L^*$ 下复用问题二模型（Skyline-ILS/SA 增量 HPWL + 终端引力合法化）更新总 HPWL 与可视化。

5.3.4 求解结果与分析

基于上述模型与算法，对附件中 n100、n200、n300 三组芯片求解最小死区比例，并在最小轮廓下更新总 HPWL，结果如表表 8 所示。其中 L_{lb} 为边长理论下界， d^* 为最小死区比例，密度 $\rho = A/(L^*)^2$ 。

表 8 问题三三组芯片的求解结果

数据集	n	L_{lb}	L^*	d^*	密度 ρ	总 HPWL@ L^*	问题 2 总 HPWL
n100	100	424	448	11.8%	89.4%	282950	276865
n200	200	420	436	8.2%	92.4%	551698	536430
n300	300	523	541	7.1%	93.3%	847564	800202

(1) 求解质量分析。三组芯片在 L^* 下均为可行布局（无重叠、不超出轮廓，CP-SAT 亦验证了面积最大的前 15 个模块在 L^* 内的可行性）。由表表 8 可见，最小死区比例分别为 11.8%、8.2%、7.1%，较问题二的死区比例 15% 显著压缩约一半；对应的轮廓密度达 89.4%–93.3%，其中 n200、n300（92.4%、93.3%）接近问题一自由轮廓实测密度，n100 略低（89.4%），与正方形轮廓长宽比固定、更难以完美铺砌的预期一致。与理论下界 d_{lb} （ $\leq 0.4\%$ ）的差距主要源于矩形模块间无法消除的铺砌间隙（空白占比约 6.7%–10.6%），符合矩形装箱的理论预期，说明”整数二分 + 双证书”在紧轮廓下仍能稳定给出高密度可行布局。

(2) 死区压缩的线长代价分析。在 L^* 下更新总 HPWL，较问题二（ $d = 0.15$ ）分别上升 2.2%、2.9%、5.9%。其机理在于：死区被压缩后，模块在轮廓内的活动空间变小，更难以在互不重叠的前提下同时逼近各自相连引脚的中位数理想位置，故总线长相应增加；其中 n300 的增幅最大（5.9%），与其密度最高（93.3%）、压缩最紧的布局状态一致。这一结果直观量化了”面积（死区）与线长”之间的权衡：最小化死区以牺

性少量线长为代价，为后续布线保留更紧凑的芯片轮廓。

(3) 收敛性分析。三组芯片在整数二分与可行性判定过程中的收敛曲线如图图 9 所示。可见各曲线呈”快速下降—缓慢逼近”形态：前期可行性 oracle 快速收缩区间，后期在临界轮廓附近逐步精调，最终稳定收敛到最小可行边长附近，说明增量收缩 warm start 与双证书策略能在有限时间内收敛到高质量可行解。

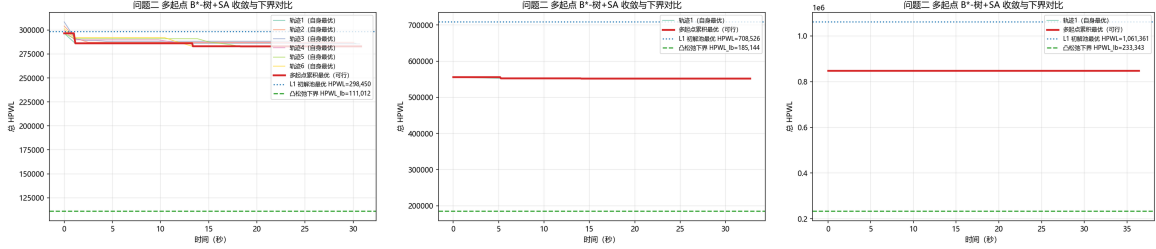


图 9 问题三三组芯片的求解收敛曲线

三组芯片在最小死区比例轮廓下的模块摆放可视化结果如图图 10 所示，图中矩形代表各功能模块，红色虚线框为 $L^* \times L^*$ 的芯片轮廓，红点为边界 Terminal。由图可见，各模块在显著缩小的正方形轮廓内仍被紧密、无重叠地排布，空白间隙细小，直观验证了最小死区比例求解结果的可行性与紧凑性。

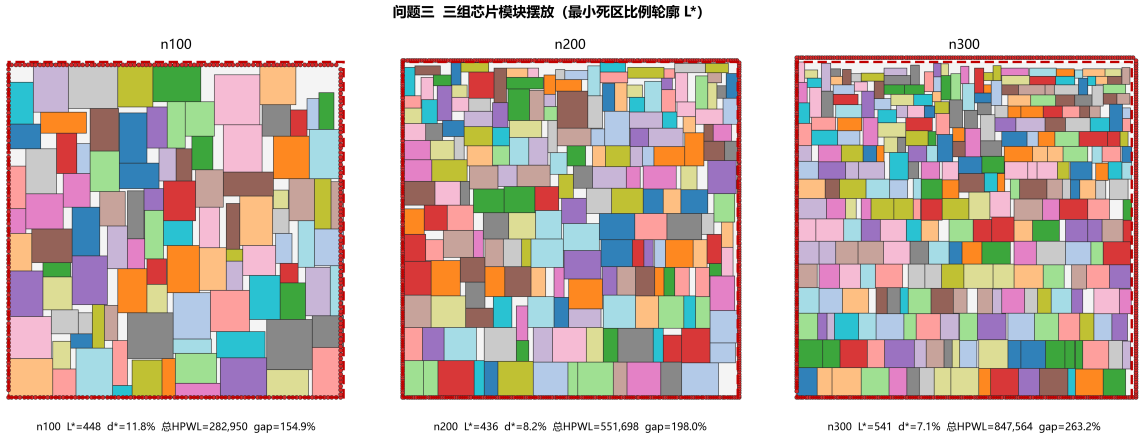


图 10 问题三三组芯片在最小死区比例轮廓下的模块摆放

5.4 问题四

第四问中需要基于这种思路和前面的研究成果，在附件 3 每个品牌中各挑选 1 名没有购买电动汽车的目标客户，实施销售策略。我们根据第三问的结果：对于品牌 1：2、4、5 号顾客有购买的意愿，1、3 号顾客没有购买的意愿；对于品牌 2：7、10 号有购买的意愿，6、8、9 号顾客没有购买的意愿；对于品牌 3：11、12 号顾客有购买的意愿，13、14、15 号顾客没有购买的意愿对模型进行优化。

5.4.1 决策树模型的优化

对于决策树模型，根据??易得品牌 1 中对于城市居住年龄具有较大的影响，而后是电池、车贷占比、房贷占比。选择品牌 1 中的 3 号顾客，相对来说城市居住年龄较为饱和。相对来说车贷，房贷比较少，具有很大的提高空间，同时，电池、经济的满意度也不高，因此建议建提高对电池，经济的满意度，各提高 2.5%，发现最后结果预测结果变为有购买意愿。

??中，品牌 2 中的 8 号顾客，在决策树中购买意愿占比 0.4，可在外观，动力，安全，经济，舒适，电池六个因素中选五个提升，根据决策树中不同因素的重要程度，选择外观、舒适、电池、经济、动力各增加 1%，发现最后结果预测结果变为有购买意愿。

??中，品牌 3 选 13 号顾客，电池，品质，外观，操控性，动力性，安全性，经济型，舒适性，八个因素中选五个进行提升，根据决策树中不同因素的重要程度，选择电池，品质，外观，操控性，动力性，安全性，经济型，舒适性各增加 0.625%，发现最后结果预测结果变为有购买意愿，符合题意。

如图 11 为模型优化后的训练精度和测试精度的图像：对于神经网络模型：根据卷

图 11 优化的决策树模型精度

神经网络模型，对其进行补充，思路为将 5% 的满意度分割成 500 份 0.01% 进行各个影响因素之间的加权，得到最终结果最大的值。

5.5 问题五

在问题一中，对电动车的经济性满意度不高是没有购买意愿人群的普遍特点，在问题二可能对购买意愿产生影响的因素中，经济性在三个品牌中均有影响；通过对不同购买意愿人群的经济状况分析可知，经济状况较好的客户更容易有购买意愿；同时，从问题三和问题四的结果可知，不同品牌、不同用户的影响因素不同。

根据以上分析，给出如下三方面销售建议：

(1) 加强购车的经济性。完善售后服务，提供保障措施，减少客户在购车时对电动车售后维修、保养等方面高成本的顾虑，提高客户对新能源汽车经济性能好的认可度。

(2) 将市场定位在较高收入人群。在新能源汽车还没有广泛普及的今天，高收入高学历、有社会责任感、倡导绿色出行的人群更容易对新能源汽车产生购买意愿，以该人群为先导，可加快新能源汽车产业的发展。

(3) 制定精准销售模型。对于不同品牌、不同人群，购买意愿有不同的影响因子，没有固定的销售模式，应该具体事例具体分析，精确分析、精确销售。

六、参考文献

参考文献

- [1] Chen J, Zhu W, Ali M M. A Hybrid Simulated Annealing Algorithm for Nonslicing VLSI Floorplanning[J]. IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews), 2011, 41(4): 544–553.
- [2] Yao B, Chen H, Cheng C K, et al. Revisiting Floorplan Representations[C]//Proceedings of the International Symposium on Physical Design (ISPD). 2001: 138–143.
- [3] Chen T C, Chang Y W. Modern Floorplanning Based on B*-Tree and Fast Simulated Annealing[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2006, 25(4): 637–652.
- [4] 黄志鹏, 李兴权, 朱文兴. 超大规模集成电路布局的优化模型与算法 [J]. 运筹学学报, 2021, 25(3): 15–36.
- [5] Adya S N, Markov I L. Fixed-Outline Floorplanning: Enabling Hierarchical Design[J]. IEEE Transactions on Very Large Scale Integration (VLSI) Systems, 2003, 11(6): 1120–1135.
- [6] Li X, Peng K, Huang F, et al. PeF: Poisson’s Equation-Based Large-Scale Fixed-Outline Floorplanning[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2023, 42(6): 2002–2015.
- [7] Mirhoseini A, Goldie A, Yazgan M, et al. Chip Placement with Deep Reinforcement Learning[J]. Nature, 2021, 594(7862): 207–212.
- [8] Zhong R, Du X, Yan J. RulePlanner: All-in-One Reinforcement Learner for Unifying Design Rules in 3D Floorplanning[C]//Proceedings of the 43rd International Conference on Machine Learning (ICML). 2026.

附录

附录 1: 问题三的 python 代码

```
# Importing the libraries
#导入包
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# Importing the dataset
#取数据
dataset =pd.read_csv('C:/Users/Administrator/Desktop/chap4.csv')
#这里取桌面的chap4的代码
#这里取D到Q列
X =dataset.iloc[:,3:16].values
#这里取S列
y =dataset.iloc[:,18].values

# Splitting the dataset into the Training set and Test set
#将数据集拆分为训练集和测试集
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test =train_test_split(X, y, test_size =0.25,
                                                    random_state =0)

# Feature Scaling

from sklearn.preprocessing import StandardScaler
sc_X =StandardScaler()
X_train =sc_X.fit_transform(X_train)
X_test =sc_X.transform(X_test)

# Fitting Classifier to the Training set
#训练集的分类器拟合
from sklearn.tree import DecisionTreeClassifier
classifier =DecisionTreeClassifier(criterion ='entropy', random_state=0,
                                   class_weight={0:1,1:10})
#这里的class_weight={0:1,1:10},max_depth=20参数可以不用加入
# classifier = DecisionTreeClassifier(criterion = 'entropy', random_state=0)
classifier.fit(X_train,y_train)

# Predicting the Test set results
#预测测试集结果
y_pred =classifier.predict(X_test)

# Making the Confusion Matrix
#制作混淆矩阵
```

```

from sklearn.metrics import confusion_matrix
cm =confusion_matrix(y_test, y_pred)

scoretest =classifier.score(X_test,y_test)

from IPython.display import Image
import pydotplus
from sklearn import tree
# 可视化决策树
dot_data =tree.export_graphviz(classifier, out_file=None,
filled=True, rounded=True,
special_characters=True)
graph =pydotplus.graph_from_dot_data(dot_data)
# 显示图片
# graph.get_nodes()[7].set_fillcolor("#FFF2DD")
graph.write_png("C:/Users/Administrator/Desktop/out.png")

*决策树.
TREE
购买意愿[n]
BY
城市居住龄[s]
/ TREE
DISPLAY =TOPDOWN
NODES =STATISTICS
BRANCHSTATISTICS =YES
NODEDEFS =YES
SCALE =AUTO
/ DEPCATEGORIES
USEVALUES =[.0 1.0]
TARGET =[1.0]
/ PRINT
MODELSUMMARY
CLASSIFICATION
RISK
/ GAIN
CATEGORYTABLE =YES
TYPE =[NODE]
SORT =DESCENDING
CUMULATIVE =NO
/ SAVE
NODEID
PREDFVAL
PREDPROB
ASSIGNMENT
/ METHOD
TYPE =CRT
MAXSURROGATES =AUTO

```

```

PRUNE =NONE
/ GROWTHLIMIT
MAXDEPTH =AUTO
MINPARENTSIZE =400
MINCHILDSIZE =200
/ VALIDATION
TYPE =SPLITSAMPLE(75)
OUTPUT =BOTHSAMPLES
/ CRT
IMPURITY =GINI
MINIMPROVEMENT =0.0001
/ COSTS
EQUAL
/ PRIORS
FROMDATA
ADJUST =NO

/ COSTS
EQUAL
COMPUTE
filter_$(品牌类型=2).
VARIABLE
LABELS
filter_$ '品牌类型=2 (FILTER)'.
VALUE
LABELS
filter_$ 0
'Not Selected'
1
'Selected'.
FORMATS
filter_$ (f1.0).
FILTER
BY
filter_$.
EXECUTE.

*决策树.
TREE
购买意愿[n]
BY
城市居住龄[s]
/ TREE
DISPLAY =TOPDOWN
NODES =STATISTICS
BRANCHSTATISTICS =YES
NODEDEFS =YES
SCALE =AUTO

```

```

/ DEPCATEGORIES
USEVALUES =[.0 1.0]
TARGET =[1.0]
/ PRINT
MODELSUMMARY
CLASSIFICATION
RISK
/ GAIN
CATEGORYTABLE =YES
TYPE =[NODE]
SORT =DESCENDING
CUMULATIVE =NO
/ SAVE
NODEID
PREDVAL
PREDPROB
ASSIGNMENT
/ METHOD
TYPE =CRT
MAXSURROGATES =AUTO
PRUNE =NONE
/ GROWTHLIMIT
MAXDEPTH =AUTO
MINPARENTSIZE =400
MINCHILDSIZE =200
/ VALIDATION
TYPE =SPLITSAMPLE(75)
OUTPUT =BOTHSAMPLES
/ CRT
IMPURITY =GINI
MINIMPROVEMENT =0.0001
/ COSTS
EQUAL
/ PRIORS
FROMDATA
ADJUST =NO

/ COSTS
EQUAL

COMPUTE
filter_$(品牌类型=3).
VARIABLE
LABELS
filter_$(品牌类型=3 (FILTER)).
VALUE
LABELS
filter_$(0

```

```

'Not Selected'
1
'Selected'.
FORMATS
filter_$ (f1.0).
FILTER
BY
filter_$.
EXECUTE.

*决策树.
TREE
购买意愿[n]
BY
城市居住龄[s]
/ TREE
DISPLAY =TOPDOWN
NODES =STATISTICS
BRANCHSTATISTICS =YES
NODEDEFS =YES
SCALE =AUTO
/ DEPCATEGORIES
USEVALUES =[.0 1.0]
TARGET =[1.0]
/ PRINT
MODELSUMMARY
CLASSIFICATION
RISK
/ GAIN
CATEGORYTABLE =YES
TYPE =[NODE]
SORT =DESCENDING
CUMULATIVE =NO
/ SAVE
NODEID
PREDVAL
PREDPROB
ASSIGNMENT
/ METHOD
TYPE =CRT
MAXSURROGATES =AUTO
PRUNE =NONE
/ GROWTHLIMIT
MAXDEPTH =AUTO
MINPARENTSIZE =400
MINCHILDSIZE =200
/ VALIDATION
TYPE =SPLITSAMPLE(75)

```

```

OUTPUT =BOTHSAMPLES
/ CRT
IMPURITY =GINI
MINIMPROVEMENT =0.0001
/ COSTS
EQUAL
/ PRIORS
FROMDATA
ADJUST =NO

/ COSTS
EQUAL

\end{appendixx}
\begin{appendixx}
\section{问题三的神经网络代码}
from predict import predictToolimport time
import cv2
import threadingimport time
from sensor_md import sensor_obj
import RPi.GPIO as GPIO
import time
from keras.datasets import mnist
import tensorflow as tf
from PIL import Image
import numpy as np
from tensorflow.keras.utils import to_categorical
from keras import layers
from keras import models
tf.compat.v1.disable_eager_execution()

model =models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
model.add(layers.MaxPool2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPool2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))
# 网络构架

model.compile(optimizer='rmsprop', loss='categorical_crossentropy', metrics=['
                                accuracy'])

# 编译步骤

# 准备图像数据

```

```

train_labels =to_categorical(train_labels)
test_labels =to_categorical(test_labels)
# 准备标签

model.fit(train_images, train_labels, epochs=5, batch_size=64)
test_loss, test_acc =model.evaluate(train_images, train_labels)
print('test_loss:%.6f' % test_loss)
print('test_acc:%.6f' % test_acc)

model.save(r'D:\Python for windows\homewor\cnn_mnist.pkl')

GPIO.setwarnings(False)GPIO.setmode(GPIO.BCM)#
GPIO.setup(4,GPIO.OUT, initial=GPIO.LOW)
GPIO.setup(12,GPIO.OUT, initial=GPIO.LOW)
GPIO.setup(13,GPIO.OUT, initial=GPIO.LOW)
GPIO.setup(16,GPIO.OUT, initial=GPIO.LOW)pin_avoid_obstacle =24
GPIO.setmode(GPIO.BCM)
GPIO.setup(pin_avoid_obstacle, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)
def thread(cap):
    global ex
    ite =0
    while True:
        global frame
        ret, frame =cap.read()
        cv2.imshow("capture", frame)
        if cv2.waitKey(100) & 0xFF ==ord('q'):
            ex =True
            break
        cap.release()
        cv2.destroyAllWindows()
    pass

if __name__ =='_main_ ':
    ex =False
    img_pre =predictTool()
    cap =cv2.VideoCapture(0)
    threa =threading.Thread(target=thread,args=(cap,))
    threa.start()
    time.sleep(2)
    global frame
    while True:
        status =GPIO.input(pin_avoid_obstacle)
        print("zhaugntai", status)
        if not status:
            time.sleep(0.5)
        _, frame =cap.read()
        resized_img =cv2.resize(frame, (224, 224), interpolation=cv2.INTER_LINEAR)

```

```

resized_img_rgb =cv2.cvtColor(resized_img, cv2.COLOR_BGR2RGB)
id =img_pre.img_predict(resized_img_rgb)
print(id)
if 0 <=id <=5:
GPIO.output(4, GPIO.HIGH)
time.sleep(1)
GPIO.output(4, GPIO.LOW)
elif 6 <=id <=12:
GPIO.output(12, GPIO.HIGH)
time.sleep(1)
GPIO.output(12, GPIO.LOW)
elif 13 <=id <=34 and id !=28:
GPIO.output(13, GPIO.HIGH)
time.sleep(1)
GPIO.output(13, GPIO.LOW)
else:
GPIO.output(16, GPIO.HIGH)
time.sleep(1)
GPIO.output(16, GPIO.LOW)
else:
time.sleep(0.5)
print("wu")
pass
if ex:
break

import os
import json
import torch
from PIL import Image
from torchvision import transforms
from models.model import resnet101import time
BASE_DIR= os.path.dirname(os.path.abspath( file__))
device =torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
class predictTool():
def _init_(self):
self.path_state_dict =os.path.join(BASE_DIR, "fine.pth" , "resNet101.pth")
self.path_classnames =os.path.join(BASE_DIR, "class_json", "class_indices.json"
                                     )self.num_classes =38
self._data_transform =transforms.Compose(
[transforms.ToTensor(),
transforms.Normalize([0.485,0.456,0.406],[0.229,0.224,0.225])])self.model =self.
                                __getModel()
self.class_indict =self._load_class_names()
pass
def _getModel(self):
model =resnet101(num_classes=self.num_classes).to(device)

```



```

assert os.path.exists(self.path_state_dict), "file: 't' dose not exist.".format(
    self.path_state_dict)
model.load_state_dict(torch.load(self.path_state_dict, map_location=device))
model.to(device)
model.eval()
return model

def img_predict(self, img):
    img =self.__img_process(img)
    with torch.no_grad():
        time_tic =time.time()
        output =torch.squeeze(self.model(img.to(device))).cpu()
        predict =torch.softmax(output, dim=0)
        predict_cla =torch.argmax(predict).numpy()
        time_toc =time.time()
        print(time_toc -time_tic) # 预测时间
        print_res ="class: prob: {:.3}.".format(self.class_indict[str(predict_cla)],
            predict[predict_cla].numpy())

    print(print_res)
    return int(predict_cla)

```

附 录

附录 1: 问题四的代码

```
import time
def _init_(self,trig,echo):
    self.trig =trig
    self.echo =echo
    GPIO.setmode(GPIO.BCM)
    GPIO.setup(trig,GPIO.OUT, initial=GPIO.LOW)
    GPIO.setup(echo, GPIO.IN)pass
def checkdist(self):
    try:
        GPIO.output(self.trig,GPIO.HIGH)
        time.sleep(0.000020)
        GPIO.output(self.trig,GPIO.LOW)
        while not GPIO.input(self.echo):
            pass
        t1 =time.time()
        while GPIO.input(self.echo):
            pass
        t2 =time.time()
    except:
        pass
    return ((t2 -t1)* 340/2)*100
if __name__=="_main_":
    while(True):
        time.sleep(0.5)
        k =sensor_obj(22,17)
        d =k.checkdist()
        print(d)
```